

// THE MCP GUIDE SERIES · BOOK ONE

Understanding MCP

What the Model Context Protocol is, why it exists, and how it lets AI assistants reach tools, data, and services — explained clearly, without assuming you already write code.

MCP Server Starter Pack

A FIELD GUIDE IN TWO PARTS

BASICS

NO CODE REQUIRED

// THE MCP GUIDE SERIES • BOOK ONE

Understanding MCP

What the Model Context Protocol is, why it exists, and how it lets AI assistants reach tools, data, and services — explained clearly, whether or not you write code for a living.

MCP Server Starter Pack

A FIELD GUIDE IN TWO PARTS • BOOK 1 OF 2

Contents

// 14 CHAPTERS • 2 APPENDICES

FRONT MATTER

—	Foreword: The New Universal Adapter	3
---	-------------------------------------	---

PART I — WHAT MCP IS

01	The Problem Before MCP	4
02	What MCP Actually Is	7
03	Three Roles: Host, Client, Server	10

PART II — THE BUILDING BLOCKS

04	Tools: Actions the Model Can Take	13
05	Resources: Data the Model Can Read	16
06	Prompts: Reusable Shortcuts	19
07	Sampling & Roots, Briefly	22

PART III — HOW IT CONNECTS

08	Local vs. Remote: The Two Transports	24
09	Trust, Permission & Safety	27

PART IV — THE ECOSYSTEM

10	Where You'll Actually Meet MCP	30
11	The Growing Server Directory	33
12	MCP vs. the Alternatives	36

PART V — GETTING STARTED

13	Using MCP Servers Day to Day	39
14	A First Taste of Building One	42

APPENDICES

A	Glossary of MCP Terms	45
B	Where to Go Next	48

The New Universal Adapter

Think about the last time you traveled internationally with a laptop, a phone, and a camera charger, and needed a small pile of plug adapters to keep them all running. Every country had picked its own socket shape, and nothing you owned matched by default. Software had the same problem with AI assistants until very recently — every assistant, every plugin system, its own shape of socket.

The Model Context Protocol, or MCP, is the universal adapter. It's an open standard that lets an AI assistant — Claude, an IDE like Cursor, a growing list of others — plug into external tools, files, databases, and services through one consistent connector, instead of a different bespoke integration for every pairing of assistant and service.

You don't need to be a programmer to understand what that unlocks. If you've ever used a browser extension, connected one app to another with Zapier, or plugged a new peripheral into a laptop and had it just work, you already understand the shape of the idea. This book explains MCP at exactly that level of clarity — what it is, why it was created, how its pieces fit together, and where you'll run into it — before any discussion of how to build one.

If you do want to build an MCP server afterward, its companion volume, *Shipping Production MCP Servers*, picks up exactly where this book leaves off and walks through building one in TypeScript, end to end. But that's a second book, for a second sitting. This one just wants you to finish it understanding MCP clearly enough to explain it to someone else over coffee.

The Problem Before MCP

Before you can appreciate a solution, it helps to feel the shape of the problem it replaced — a tangle of one-off integrations that grew worse with every new tool and every new assistant.

THIS CHAPTER COVERS

- The N×M integration problem
- Why plugins didn't fix it
- What "protocol" buys you
- The moment MCP was introduced

The Problem Before MCP

Picture the wiring closet

Imagine a company with four AI assistants in use across different teams, and six internal systems each of them might reasonably need to reach — a customer database, a ticketing system, a file store, an internal search index, a calendar, and a billing API. Before MCP, connecting all of them meant building a separate integration for every assistant-and-system pair. Four assistants times six systems is twenty-four integrations, each with its own authentication quirks, its own data format, its own bugs to fix twice — or four times.

4 × 6

ASSISTANTS × SYSTEMS

24

CUSTOM INTEGRATIONS NEEDED

1INTEGRATION, WITH A SHARED
PROTOCOL

This is what engineers call the N×M problem: the number of integrations you need grows as the product of how many things need to connect, not the sum. Add a fifth assistant, and you don't add one integration — you add six. Add a seventh system, and you add four more. The tangle gets worse exactly when adoption is going well, which is the cruelest part of this kind of problem.

Why plugin systems didn't solve it

Individual AI products did try to solve part of this. Several shipped their own plugin or "actions" systems, letting developers register a tool that one specific assistant could call. That helped — for that one assistant. A plugin built for one product's format didn't run in a competitor's product, so a developer supporting three assistants was still maintaining three separate integrations that did roughly the same thing in three incompatible shapes.

A standard only pays off once more than one party agrees to speak it. One company's plugin format is still just a private dialect, no matter how good it is.

What "protocol" actually buys you

A protocol is an agreement about the shape of a conversation, independent of who's having it. Email works because every mail provider speaks SMTP; the web works because every browser and server speaks HTTP. Neither of those required Gmail and Outlook to merge, or Chrome and Safari to share code — they just agreed on the same rules for exchanging messages.

MCP does the same job for AI assistants and the tools they use. Once a system speaks MCP, any MCP-compatible assistant can use it — and once an assistant speaks MCP, it can use any MCP-compatible system. The $N \times M$ problem collapses into an $N+M$ problem: each side builds one integration, to the protocol, instead of one per pairing.

Where MCP came from

Anthropic introduced the Model Context Protocol in late 2024 as an open standard — not a proprietary Claude feature, but a specification anyone could implement, with a public specification and open-source SDKs. That openness is why adoption spread beyond Claude itself into other AI products relatively quickly: the whole value of a protocol depends on more than one party using it, so giving it away was the strategy, not a concession.

What MCP Actually Is

A precise, jargon-light definition of the protocol itself — what it standardizes, what it's built on top of, and what it deliberately leaves open.

THIS CHAPTER COVERS

- A one-sentence definition
- The message format underneath
- What MCP standardizes
- What MCP leaves up to you

What MCP Actually Is

One sentence

MCP is an open protocol that lets an AI assistant discover and call external tools, read external data, and use shared prompt templates, through one standard interface that works the same way regardless of which assistant or which tool is on the other end.

Every other detail in this book is an elaboration of that one sentence. It's worth re-reading before continuing, because almost every question people have about MCP ("does it replace X," "is it an API," "do I need to learn a new language") is answered by holding this definition steady and checking the question against it.

Built on a well-worn message format

MCP messages are JSON-RPC 2.0 — a lightweight, decades-old convention for describing a remote procedure call as a small JSON object with a method name, some parameters, and an ID to match requests to responses. MCP didn't invent a new wire format; it reused a proven one and defined a specific vocabulary of methods on top of it.

A TOOLS/CALL REQUEST, SHOWN FOR INTUITION — NOT SOMETHING YOU NEED TO WRITE BY HAND

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "get_forecast",
    "arguments": { "city": "Lisbon" }
  }
}
```

You'll likely never write this JSON by hand — the SDKs and client applications handle it — but seeing its shape once demystifies a lot. It's a named method, some arguments, and an ID. Nothing more exotic is happening under the hood of a "tool call."

What MCP standardizes

- **Discovery** — a consistent way for a client to ask a server "what can you do?" and get back a structured list of tools, resources, and prompts.
- **Invocation** — a consistent way to call a tool with arguments and get a result back, including a consistent shape for errors.
- **Data access** — a consistent way to address and read external data by URI.
- **Capability negotiation** — client and server agree at connection time on which optional features (like sampling, covered in Chapter 7) they both support.

What MCP deliberately leaves open

MCP doesn't dictate what a tool actually does behind the scenes, what language a server is written in, how a server authenticates its own users, or how a company chooses to host one. It standardizes the conversation, not the implementation — the same restraint HTTP shows toward what a website is allowed to contain.

STANDARDIZED BY MCP	LEFT UP TO THE IMPLEMENTER
Message format & method names	Programming language & framework
How tools/resources/prompts are described	What a tool actually does internally
Discovery & invocation flow	Hosting, scaling, and deployment choices
Optional-feature negotiation	Business logic and data sources

Three Roles: Host, Client, Server

Every MCP conversation involves exactly three kinds of participant. Once you can name which is which in any setup you encounter, the rest of the architecture falls into place quickly.

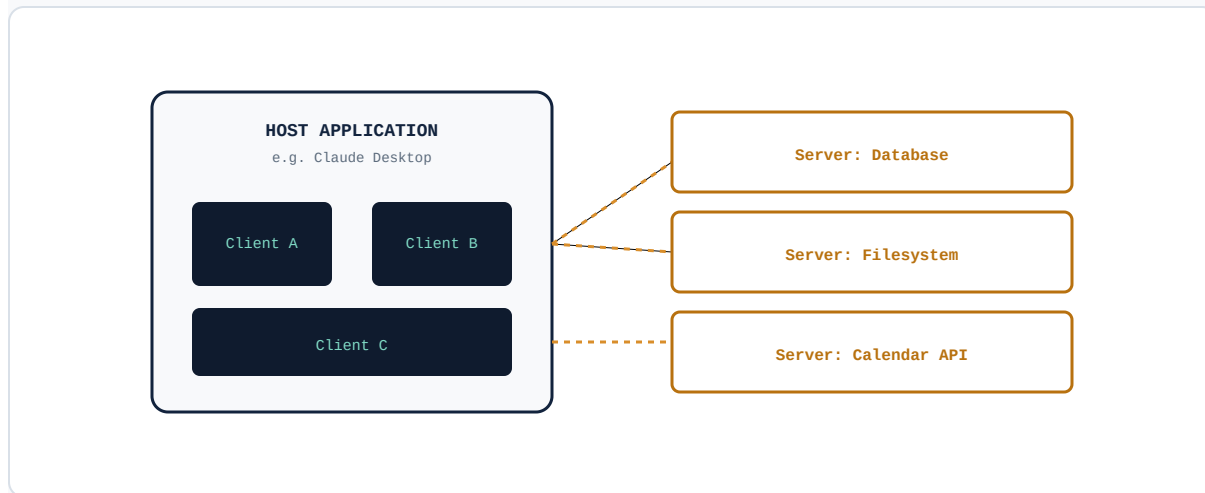
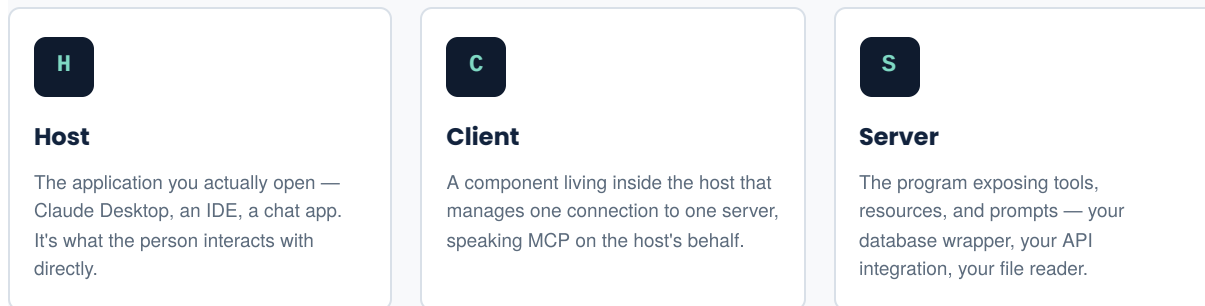
THIS CHAPTER COVERS

- The host application
- The client, inside the host
- The server, outside the host
- One host, many servers

Three Roles: Host, Client, Server

The cast of characters

Every piece of MCP terminology you'll encounter maps onto one of three roles. Keeping them straight resolves most of the confusion newcomers run into.



One host, many servers

A single host application typically runs several clients at once, one per server it's connected to — Claude Desktop might hold an open connection to a filesystem server, a database server, and a calendar server simultaneously, each managed by its own client instance. That's why you'll sometimes see "client" and "host" used loosely as if they're the same thing; in casual conversation they usually are, but the distinction matters the moment you're troubleshooting a specific connection rather than the application as a whole.

Servers don't know about each other

An important, easy-to-miss property: servers are isolated from one another. The database server in the diagram above has no idea the calendar server exists, and can't call it directly. Only the host — through the model's reasoning — decides to use information from one server as input to a call on another. This isolation is a deliberate security boundary, not an oversight, and it's the topic Chapter 9 covers in full.

A USEFUL MENTAL SHORTCUT

Host = the app you opened. Client = the wire inside it. Server = the thing on the other end of the wire. If you can point to each of the three in whatever setup you're looking at, you understand the architecture.

Tools: Actions the Model Can Take

The primitive that does the most work in practice: a named, described action the model can decide to invoke, with arguments and a result.

THIS CHAPTER COVERS

- What makes something a "tool"
- Common categories of tools
- How the model decides to use one
- Why descriptions matter so much

Tools: Actions the Model Can Take

The verb primitive

If resources (next chapter) are nouns — things the model can read — tools are verbs: actions with a side effect or a computation behind them. Send an email, run a search, create a calendar event, query a database, convert a file. Anything you'd describe with an active verb is a tool candidate.

How the model decides to call one

When a host connects to a server, the client asks it "what tools do you have?" and gets back a list — each tool's name, a plain-language description of what it does, and a description of what arguments it takes. That list becomes part of what the model sees while it's reasoning about how to help you. If you ask an assistant to "email Sarah the quarterly numbers," and a `send_email` tool is available with a description that mentions sending emails, the model can recognize that connection and decide to call it — the same way a person scanning a toolbox recognizes which tool fits the job in front of them.

WHY THE DESCRIPTION MATTERS SO MUCH

The model never sees the code behind a tool — only its name and description. A vaguely described tool gets used incorrectly or ignored entirely, no matter how well it's implemented. This is why Chapter 3 of the companion volume treats schema-writing as seriously as the underlying logic: the description *is* the interface, from the model's point of view.

Common categories



Read & fetch

Query a database, search an index, look up a record — retrieving information the model can reason about.



Write & act

Send a message, create a ticket, update a record — actions with a real side effect out in the world.



Compute & transform

Convert a file format, run a calculation, generate a chart — pure computation with no external state.

Write-and-act tools deserve the most caution: unlike a read, they can't always be undone, so servers exposing them typically add confirmation steps or narrower permissions than a read-only tool would need — a theme Chapter 9 returns to.

A tool call, end to end

1. You ask a question
2. Model decides a tool fits
3. Client calls the server's tool
4. Result returns to the model

"What's the weather in Lisbon?"

`get_forecast(city: "Lisbon")`

`tools/call → server`

"Sunny, 22°C" → reply to you

Resources: Data the Model Can Read

The noun primitive: addressable, read-only data the client can pull in as context — closer to a file browser than a search engine.

THIS CHAPTER COVERS

- Resources vs. tools
- URIs as addresses
- Static vs. dynamic resources
- A worked example

Resources: Data the Model Can Read

The noun primitive

Where a tool is something the model actively decides to call, a resource is something the host can present as available context — a file, a database record, a configuration value, a log — addressed by a URI, the same way a web address points at a specific page rather than a whole website.

	TOOLS	RESOURCES
Shape	Verb — an action with arguments	Noun — addressable data
Who decides to use it	The model, during reasoning	Often the person, browsing what's available
Side effects	Can read or write	Read-only, by design
Familiar analogy	A function call	A file path or URL

Addressing with URIs

A resource's URI can look like a familiar web address, a file path, or something entirely custom to the server — `file:///reports/q3.pdf` , `postgres://orders/12345` , `github://repo/issues/42` . The scheme just needs to be something the server understands; MCP doesn't mandate a specific set of schemes.

Static vs. dynamic resources

Some resources are fixed — a specific file, a specific configuration document. Others are templated: a server can expose a pattern like `orders/{id}` and let the client fill in the ID, generating an address for a record that didn't exist as a distinct listing beforehand. This is closer to a REST API's path parameters than to a static file listing, and it's how most database-backed resources work in practice.

A worked example

Picture a documentation server exposing your team's internal wiki. Each page becomes a resource — `wiki://onboarding/day-one` , `wiki://engineering/deploy-process` — and the host can list them, let a person browse and attach a relevant page to the conversation, or let the model pull one in automatically if a tool call or the conversation makes clear which page is relevant. Either way, the assistant is reading the same page a person would find by clicking through the wiki — just addressed through MCP instead of a browser.

RESOURCES DON'T REPLACE TOOLS

A server exposing "read the customer record" as a resource and "update the customer record" as a tool isn't being redundant — it's using each primitive for what it's for. Resources are what gets read; tools are what gets done. A well-designed server usually has both.

Prompts: Reusable Shortcuts

The primitive built for people, not just models: a saved, parameterized prompt template that a server can offer as a one-click shortcut.

THIS CHAPTER COVERS

- What a prompt primitive is
- Why it's not just a tool
- Parameters and templates
- A worked example

Prompts: Reusable Shortcuts

A saved query, not a saved answer

A prompt in MCP terms is a template — a piece of pre-written instruction, often with a few blanks to fill in, that a server offers so a person doesn't have to retype a complex request every time they need it. Think of it like a saved search or a template email: the structure is fixed, a few details change each time you use it.

Why this isn't just a tool

Tools are things the *model* decides to call mid-conversation. Prompts are usually surfaced to the *person* directly — often as a slash command or a menu item in the host application — as a way to kick off a conversation with a well-formed request, rather than something invoked automatically while the model is reasoning. The distinction is about who's driving: a tool call is the model's decision; a prompt selection is typically yours.

A worked example

A code-review server might expose a prompt called `review_pull_request` that takes a PR number as its one parameter and expands into detailed reviewer instructions — check for security issues, verify test coverage, flag breaking API changes — without you needing to type that checklist out by hand every time you ask for a review. Select the prompt, provide the PR number, and the fully-formed instruction goes to the model as your message.

```
/review_pr  
pr_number: 142
```



```
"Review PR #142 for security  
issues, test coverage, and  
breaking API changes..."
```

LEAST-USED, BUT NOT LEAST VALUABLE

Prompts get far less attention than tools in most discussions of MCP, largely because tools do the more visible work. But for teams standardizing how their whole org asks an assistant to do something — a consistent incident-report format, a consistent code-review checklist — prompts are the primitive built exactly for that.

Sampling & Roots, Briefly

Two less common primitives, worth knowing by name even though you'll encounter them far less often than tools, resources, and prompts.

THIS CHAPTER COVERS

- Sampling — servers borrowing the model
- Roots — scoping what a server can see
- Why both are optional

Sampling & Roots, Briefly

Sampling: servers borrowing the model

Every primitive so far has the model calling the server. Sampling flips that: it lets a *server* ask the *host's* model to generate something on its behalf, mid-task. A summarization server processing a long document might use sampling to ask the model to condense a section, rather than shipping its own separate AI model inside the server. It borrows the intelligence already sitting in the host instead of duplicating it.

This is opt-in on both sides — a host has to support offering sampling, and a person typically approves each sampling request, since it means a server is asking the assistant to generate content rather than the person asking directly.

Roots: scoping what a server can see

Roots let a client tell a server which directories or URIs are actually in scope for a given session — "you may work within this project folder," rather than the server assuming access to an entire filesystem. A code-editing host might set the currently open project folder as the only root, so a filesystem server connected to it structurally can't wander outside that folder, rather than merely being asked politely not to.

WHY THESE TWO GET LESS AIRTIME

Both are genuinely useful, and both are optional features that a client and server negotiate at connection time — many day-to-day MCP setups never touch either one. They're included here so the terms aren't a mystery if you encounter them, not because you need them to use or build a basic MCP server.

Local vs. Remote: The Two Transports

How the client and server actually exchange messages — over a process pipe on your own machine, or over the network to a hosted service.

THIS CHAPTER COVERS

- stdio, for local tools
- HTTP, for hosted servers
- Why the distinction matters to you
- How to tell which you're using

Local vs. Remote: The Two Transports

Same conversation, two different wires

Everything in Part II — tools, resources, prompts — describes *what* gets said between a client and server. Transport is about *how* the message physically gets from one to the other, and MCP supports two.

L

stdio (local)
The host starts the server as a program running on your own computer and talks to it directly. No network involved — like launching an app.

R

HTTP (remote)
The server runs elsewhere — on a company's servers, in the cloud — and the host talks to it over the internet, the same way a browser talks to a website.

Why this distinction actually matters to you

It shapes what you should expect and where you should look when something's wrong. A local (stdio) server is only as available as your own machine — if it's not running, nothing on the internet is broken, you just need to check whether the program started. A remote (HTTP) server depends on someone else keeping a service online, the same as any website you rely on; if it's down, it's down for everyone connected to it, not just you.

	STDIO (LOCAL)	HTTP (REMOTE)
Runs where	Your own computer	A server elsewhere, reached over the internet
Typical use	Claude Desktop, local IDE tools	Hosted, shared, or multi-user services
Needs an internet connection	No	Yes
Who can use it	Just you, on that machine	Anyone the server is configured to allow

How to tell which you're using

If setting up a server involved installing something on your computer and pointing a configuration file at a local program, you're using stdio. If it involved signing in, authorizing access through a login screen, or entering a URL, you're using the HTTP transport talking to a server hosted somewhere else. Chapter 13 walks through both setup flows concretely.

YOU RARELY HAVE TO CHOOSE

As someone using MCP servers rather than building them, the transport is usually decided for you by whoever built the server — you'll follow whatever setup instructions they provide. The distinction matters most once you're the one deciding how to build and ship a server, which the companion volume covers in depth.

Trust, Permission & Safety

Connecting an assistant to real tools and real data raises real questions. Here's how MCP's trust model addresses them, and what you should still check yourself.

THIS CHAPTER COVERS

- Why this matters more than a typical app install
- The consent & approval model
- Least privilege in practice
- What to check before connecting a server

Trust, Permission & Safety

A different kind of install

Connecting an MCP server isn't quite like installing a normal app. You're giving an AI assistant the ability to act — sometimes to read sensitive data, sometimes to take actions like sending messages or modifying records — based on its own judgment about what a conversation calls for. That's a meaningfully different trust decision than installing a calculator app, and the protocol's design reflects that.

The consent model

Well-behaved host applications are built to keep a person in the loop at two key moments: when a server is first connected (you typically see and approve what it can access), and often again when a tool is about to do something consequential, like sending an email or deleting a record. The specifics vary by host application, but the underlying principle is consistent — the assistant shouldn't be silently taking actions you never agreed to let it take.

Least privilege in practice

A well-designed server asks for only the access it actually needs. A tool that reads your calendar shouldn't also request the ability to delete files; a server built for one team's ticketing system shouldn't request access to your entire company's customer database. When you're choosing between two servers that do similar things, the one asking for narrower access is usually the better-designed one, not the less capable one.

What to actually check before connecting a server

1. **Who published it.** A server from a company whose product it integrates with, or a well-known open-source maintainer, carries more accountability than an anonymous listing.
2. **What it asks to access.** Does the requested scope match what the server claims to do? A "read my calendar" server asking for filesystem access is a mismatch worth questioning.
3. **Whether it's actively maintained.** An abandoned server is a server whose security issues, if any turn up, will never get fixed.

4. **Where credentials go.** A trustworthy server's setup instructions never ask you to paste a password or API key somewhere that isn't the official config file or a recognized secrets manager.

THE SAME INSTINCT AS INSTALLING A BROWSER EXTENSION

If you already have a habit of checking a browser extension's permissions and publisher before installing it, you already have the right instinct for evaluating an MCP server. The stakes and the checklist are genuinely similar.

Where You'll Actually Meet MCP

A tour of the applications that speak MCP today, so you know where to look for it whether or not you ever build a server yourself.

THIS CHAPTER COVERS

- Desktop AI assistants
- Code editors & IDEs
- Enterprise & team tools
- What to expect next

Where You'll Actually Meet MCP

Desktop AI assistants

Claude Desktop was the first major host application to support MCP and remains one of the most complete implementations — connecting local servers is a matter of editing a configuration file or, increasingly, a few clicks in a settings panel. Other desktop assistants have added support since, following the same open specification.

Code editors & IDEs

This is where MCP adoption has moved fastest. Cursor, VS Code (through GitHub Copilot's agent mode), and several other AI-assisted editors support MCP servers as a way to give the assistant access to project-specific context — a design system, an internal API, a issue tracker — beyond what's in the open files.

CATEGORY	EXAMPLES	TYPICAL USE
Desktop assistants	Claude Desktop	Personal productivity, local file & tool access
Code editors	Cursor, VS Code (Copilot)	Project context, internal APIs, dev tooling
Team & enterprise tools	Internal chat assistants, support tools	Company-specific data and workflows

Enterprise & team tools

Companies building internal AI assistants — for customer support, internal knowledge search, or workflow automation — increasingly expose their internal systems as MCP servers rather than building bespoke integrations for whichever assistant they've adopted. This is the same N+M economics from Chapter 1, applied inside a single organization: build the server once, and it works with whatever MCP-compatible assistant the company uses today or switches to later.

What to expect next

The pattern across every category above is the same: whichever product you already use is likely to add MCP support if it hasn't already, because doing so is now the default way to let an AI feature reach beyond its own walls. Expect the list of MCP- compatible hosts to keep growing faster than any book can stay current about — which is exactly why the next chapter points you toward living directories instead of a fixed list.

The Growing Server Directory

Thousands of MCP servers already exist for common tools and services. Here's where to actually find them.

THIS CHAPTER COVERS

- Community server directories
- Evaluating a listing before you install
- The official reference servers

The Growing Server Directory

You rarely need to build one to benefit

A common misconception is that using MCP means writing a server yourself. In practice, most people benefit from MCP entirely as a user: connecting servers other people or companies have already built for popular tools — search engines, databases, project trackers, note-taking apps, version control platforms — the same way most web users have never written a website but benefit from millions that already exist.

Where to find them

- **mcp.so** — a community-run directory organized by category, one of the most actively updated listings of public servers.
- **mcpservers.org** — another community directory with search and filtering by the kind of integration you need.
- **Smithery** — a directory that also handles hosting and installation for many listed servers, reducing setup to a few clicks.
- **The official reference servers** — a small set of servers maintained directly alongside the MCP specification, covering common cases like filesystem access and version control, useful both as working tools and as examples of well-formed servers.

Evaluating a listing before you install

Directories vary in how much they vet what's listed — treat a listing as a starting point for your own check, not a guarantee. Revisit Chapter 9's checklist (publisher, requested scope, maintenance activity, credential handling) for anything you're about to connect, especially anything touching sensitive data or the ability to take actions on your behalf.

THIS ECOSYSTEM MOVES FAST

The specific directories above are a snapshot, not a permanent list — new ones appear and existing ones add features regularly. Search for "MCP server directory" if you're reading this well after publication and want the current landscape rather than this chapter's specific names.

MCP vs. the Alternatives

How MCP relates to function calling, proprietary plugin systems, and agent frameworks like LangChain — three things it's often confused with, for good reason.

THIS CHAPTER COVERS

- MCP vs. function calling
- MCP vs. agent frameworks
- MCP vs. proprietary plugins
- How they actually fit together

MCP vs. the Alternatives

These aren't really competitors — they're different layers

A lot of confusion here comes from comparing things that solve different problems. This chapter untangles the three comparisons people ask about most.

COMPARED TO	WHAT IT ACTUALLY IS	HOW IT RELATES TO MCP
Function calling	A model capability — the model's ability to output a structured call to a named function	MCP tools are delivered <i>to</i> the model using function calling underneath; MCP standardizes where that function list comes from
Proprietary plugins	A vendor-specific integration format, tied to one AI product	Solves the same problem for one assistant only; MCP solves it once, for every compatible assistant
Agent frameworks (e.g. LangChain)	A developer library for building multi-step AI applications and chaining tool calls together	A framework can use MCP servers as its source of tools, the same way it might otherwise use hand-written functions — the two work together, not against each other

Function calling: the engine, not the fuel line

Function calling is what lets a model output "call this function with these arguments" in a structured way — it's a capability of the model itself. MCP doesn't replace that; it standardizes how the list of available functions gets assembled and delivered to the model in the first place, and how the result gets handled afterward. Function calling is the engine; MCP is closer to the standardized fuel line feeding it from any compatible tank.

Plugins: solved, but only once

A proprietary plugin system genuinely solves the integration problem — for the one product that built it. The limitation isn't technical, it's that the work doesn't transfer. An assistant built by a different company, or even a different tool made by the same company later, doesn't automatically benefit from a plugin someone built for the first one.

Agent frameworks: a different job entirely

LangChain, and frameworks like it, help developers build applications that chain multiple AI calls and tool uses together into a larger workflow — retrying, branching, combining results. That's an application-building concern. MCP is about where the tools those chains use come from. A LangChain application can absolutely call out to MCP servers for its tools instead of hand-writing each integration, combining both without conflict.

MCP answers "how does a tool get to the model," not "what does the model do once it has one." Keeping those two questions separate resolves most of the "is MCP the same as X" confusion.

Using MCP Servers Day to Day

A concrete walkthrough of connecting your first server — whether it's a local tool on your own machine or a hosted, remote one.

THIS CHAPTER COVERS

- Connecting a local server
- Connecting a remote server
- What approval prompts to expect
- Troubleshooting the basics

Using MCP Servers Day to Day

Connecting a local (stdio) server

Most desktop host applications follow the same basic shape: a settings screen or a configuration file where you add an entry naming the server and how to start it. In Claude Desktop, for instance, this typically means adding a short block to a configuration file naming the server and the command that launches it — often a single line copied directly from the server's own setup instructions.

1. Install the server's dependencies, following its own README (usually one or two commands).
2. Add the server's entry to your host application's configuration, exactly as its setup guide describes.
3. Restart the host application so it picks up the new configuration.
4. Look for the server's tools to appear in the assistant's available-tools list or a small icon indicating a successful connection.

Connecting a remote (HTTP) server

A hosted server usually involves less local setup and more of an authorization flow: you're typically given a URL to add in the host's settings, then redirected through a login screen — the OAuth flow described conceptually in the companion volume's Chapter 5 — to grant the connection access to whatever account it needs.

What approval prompts to expect

The first time a newly connected server's tool is about to run, well-built host applications show you what's about to happen and ask for confirmation, particularly for anything that writes or sends rather than just reads. Expect this to happen more often right after connecting a server and less often once you've established a pattern of trust with it — many hosts let you approve a tool "always" for a session or permanently, once you're comfortable with what it does.

Troubleshooting the basics

SYMPTOM	LIKELY CAUSE
Server doesn't appear after setup	Configuration file syntax error, or the host wasn't restarted
Tools listed but calls fail	Missing or expired credentials for the underlying service
Assistant doesn't use the tool when expected	The request didn't clearly match the tool's description — try rephrasing more directly
Remote server connection drops	Usually transient — most hosts reconnect automatically within a session

A First Taste of Building One

Not a full tutorial — just enough to see that a minimal MCP server is a small, approachable piece of code, before this book hands off to its companion volume.

THIS CHAPTER COVERS

- What building one actually involves
- A minimal server, briefly shown
- What production servers add on top
- Where to go from here

A First Taste of Building One

Smaller than it sounds

If everything so far has sounded like heavyweight infrastructure, the actual starting point is reassuringly small. An SDK exists for MCP in most popular languages — TypeScript, Python, and others — and each one handles the JSON-RPC plumbing from Chapter 2 for you. Building a minimal server mostly means describing one tool and writing the ordinary function it calls.

```
A COMPLETE, MINIMAL SERVER — THIS IS GENUINELY MOST OF IT
const server = new McpServer({ name: "weather", version: "1.0.0" });

server.tool("get_forecast", "Get the weather forecast for a city",
  { city: z.string() },
  async ({ city }) => ({
    content: [{ type: "text", text: await fetchForecast(city) }]
  })
);
```

Name the server, describe one tool, write the plain function behind it. That's a working MCP server a host application can connect to today.

What separates this from a production server

What that snippet doesn't show is everything a server needs before you'd hand it to a teammate or put it in front of real users: validating input carefully, authenticating who's allowed to call it, limiting how often it can be called, logging failures usefully instead of silently, testing it, containerizing it, and deploying it somewhere reliable. None of that changes the core shape above — it wraps around it, the same way a production Express API wraps around a simple route handler.

~10

LINES FOR A WORKING TOOL

14

CHAPTERS OF PRODUCTION HARDENING

1

COMPANION VOLUME COVERING ALL OF
IT

Where this book hands off

Everything past this point — authentication, rate limiting, structured error handling, testing, Docker, deployment, CI/CD, and a set of real integration examples — is exactly what *Shipping Production MCP Servers*, this book's companion volume, covers in full, with working TypeScript for every piece. If the small snippet above made you want to see the whole path from here to something you'd actually ship, that's the book to open next.

Glossary of MCP Terms

Every term from this book, in one place, for quick lookup later.

THIS APPENDIX COVERS

- Core architecture terms
- The three primitives
- Transport & connection terms
- Advanced & ecosystem terms

Glossary of MCP Terms

MCP CORE

The Model Context Protocol — an open standard letting AI assistants connect to external tools, data, and prompt templates through one consistent interface.

Host ARCHITECTURE

The application you interact with directly — Claude Desktop, an IDE, a chat app.

Client ARCHITECTURE

A component inside the host managing one connection to one server.

Server ARCHITECTURE

The program exposing tools, resources, or prompts to a connected client.

Tool PRIMITIVE

A named, described action the model can decide to call, with arguments and a result — the "verb" primitive.

Resource PRIMITIVE

Read-only data addressed by URI that a client can fetch — the "noun" primitive.

Prompt PRIMITIVE

A reusable, parameterized template a server offers, typically selected directly by the person rather than the model.

Sampling ADVANCED

An optional feature letting a server ask the host's model to generate content on its behalf.

Roots ADVANCED

An optional feature letting a client scope which directories or URIs a server may work within.

JSON-RPC 2.0 TRANSPORT

The lightweight message format MCP is built on — a JSON object naming a method, its parameters, and a request ID.

stdio TRANSPORT

The local transport: host and server communicate as a parent and child process on the same machine, no network involved.

HTTP + SSE TRANSPORT

The remote transport: host and server communicate over the network, the way a browser talks to a website.

OAuth 2.1 SECURITY

The authorization standard MCP's remote-server spec builds on for letting a server act on behalf of a specific signed-in user.

Scope SECURITY

A defined, limited slice of access — "read-only," "calendar access only" — granted to a key or a connection rather than blanket access.

N×M problem CONCEPT

The integration-cost problem MCP solves: without a shared protocol, the number of custom integrations needed grows as assistants multiplied by tools, not added.

Where to Go Next

Resources for going deeper, and the natural next step once you're ready to build.

THIS APPENDIX COVERS

- Official docs & specification
- Server directories
- The companion volume
- Communities

Where to Go Next

OFFICIAL DOCS & SPECIFICATION

- The Model Context Protocol specification & documentation — modelcontextprotocol.io
- Reference SDKs for TypeScript, Python, and other languages, maintained alongside the spec

SERVER DIRECTORIES

- mcp.so and mcpservers.org — community-run directories of public servers
- [Smithery](https://smithery.io) — a directory that also handles hosting and one-click installation

READY TO BUILD ONE?

THE COMPANION VOLUME

Shipping Production MCP Servers — Book 2 in this series — picks up exactly where Chapter 14 left off: authentication, rate limiting, structured error handling, testing, Docker, deployment, CI/CD, and four full real-world integration examples, all in working TypeScript for developers coming from a Node or Express background.

COMMUNITIES

- Cursor and Claude Code communities on Discord and Reddit ([r/cursor](https://r.cursor.com)), where MCP setup and new servers are discussed daily
- GitHub Discussions on the official MCP specification repository, for protocol-level questions